ESTRUCTURAS DE DATOS Y ALGORITMOS

Búsqueda y ordenamiento

Programador Universitario en Sistemas
Primer Año - 2° cuatrimestre

Buscar y ordenar

Buscar es averiguar si un dato está en una colección y, si está, en qué posición. La respuesta es un índice o un valor especial (-1) que significa «no está».

Ordenar es reacomodar los elementos según un criterio (de menor a mayor, alfabéticamente, por fecha…) para que cada uno quede en la posición que le corresponde.

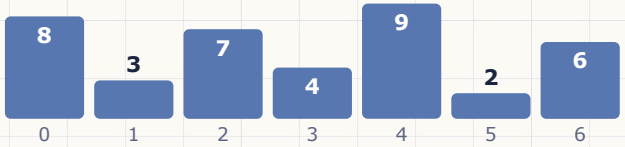
¿Por qué van juntas? Porque tener los datos ordenados permite buscar mucho más rápido. Casi todo programa que maneja información hace las dos cosas todo el tiempo.

Datos de ejemplo. Para esta clase usamos el mismo arreglo de 7 números, para poder comparar las estrategias entre sí:

Datos: 8 3 7 4 9 2 6

Ordenados: 2 3 4 6 7 8 9

Datos



Ordenados



Cómo leer los colores

- sin tocar
- comparando
- intercambio o pivote
- en su lugar / encontrado
- descartado

Cómo medimos el costo de un algoritmo

n es la cantidad de datos. Un algoritmo se mide por cuánto trabajo hace en función de n, contando operaciones (casi siempre comparaciones) y no segundos: el tiempo depende de la máquina, las operaciones no.

Mejor y peor caso. El mejor caso es la situación más favorable (por ejemplo, que lo buscado esté primero). El peor caso es la más desfavorable (que esté último o que no esté). Se suele analizar el peor caso porque es una garantía: «como mucho hace esto».

Notación O grande. Describe cómo crece el trabajo cuando n crece, sin fijarse en constantes:

O(n): si los datos se duplican, el trabajo se duplica.

O(n²): si los datos se duplican, el trabajo se cuadruplica.

O(log n): si los datos se duplican, se suma una sola operación más.

O(n log n): crece algo más que n, pero muchísimo menos que n².

Qué pasa si n se duplica

Costo	n = 1.000	n = 2.000
O(n)	1.000	2.000
O(n²)	1.000.000	4.000.000
O(log n)	10	11
O(n log n)	≈ 10.000	≈ 22.000

Cantidad de operaciones aproximada, en el peor caso.

https://www.geeksforgeeks.org/dsa/what-does-big-o-olog-n-complexity-mean/

Búsqueda secuencial: el código

for recorre las posiciones en orden, de 0 a n − 1.

if compara el valor de esa posición con el buscado.

return i devuelve el índice (la posición, no el valor) apenas lo encuentra.

return -1 solo se ejecuta si el for terminó sin éxito. Se usa -1 porque ninguna posición de un arreglo puede valer -1: es una forma segura de decir «no está».

Con objetos. == compara referencias («¿es el mismo objeto?»); equals compara contenido («¿dicen lo mismo?»). Con Strings u objetos se usa equals: datos[i].equals(buscado).

Cuándo usarla. Cuando los datos no tienen orden, o son pocos: es simple y siempre correcta.

Busqueda.java

```java
int buscar(int[] datos, int
buscado) {
  for (int i = 0; i < datos.length; i++)
{
    if (datos[i] == buscado) {
      return i;
    }
  }
  return -1;
}
```

Búsqueda secuencial: ¿cuánto trabajo hace?

Contamos comparaciones sobre el arreglo de 7 elementos 8 3 7 4 9 2 6:

Buscar 8 (está primero): 1 comparación → mejor caso.

Buscar 4: 4 comparaciones.

Buscar 6 (está último): 7 comparaciones → peor caso.

Buscar 5 (no está): 7 comparaciones → peor caso, porque hay que revisar todo para estar seguros.

Complejidad. Mejor caso O(1); peor caso O(n). Cuando decimos que la búsqueda secuencial es O(n) nos referimos al peor caso: con 1.000 datos, hasta 1.000 comparaciones; con 1.000.000, hasta un millón.

Promedio. Si el valor está y es igual de probable en cualquier posición, se revisan en promedio unos n/2 elementos. Sigue siendo O(n), porque el trabajo crece en proporción a n.



Búsqueda binaria: la idea

Idea. En vez de revisar uno por uno, se compara con el elemento del medio y, según el resultado, se descarta la mitad del tramo. Es como adivinar un número del 1 al 100 con «mayor» o «menor», o abrir la guía telefónica por la mitad.

Condición. Los datos tienen que estar ordenados.

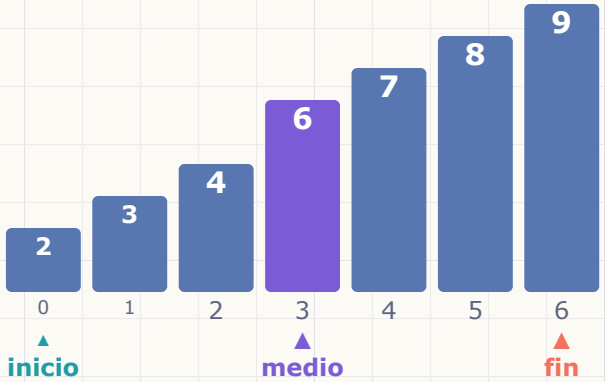
Tres índices:

inicio y fin encierran el tramo donde todavía puede estar el valor (al principio, todo el arreglo).

medio es la posición que se mira: inicio + (fin − inicio) / 2.

En cada vuelta. Si datos[medio] es igual al buscado, se termina. Si es menor, el valor está a la derecha (inicio = medio + 1). Si es mayor, está a la izquierda (fin = medio − 1).

Buscamos el 7 en un arreglo ordenado



Binaria: por qué exige datos ordenados

Por qué. Mirar el centro permite descartar una mitad solo si sabemos que todo ese lado es mayor (o menor) que lo buscado. Sin orden, el valor del medio no dice nada sobre el resto.

Con datos desordenados. Buscamos el 2 en 8 3 7 4 9 2 6. El medio es 4; como 2 < 4, el algoritmo descarta la derecha… pero el 2 estaba en el índice 5. Resultado: «no está», que es incorrecto.

Con datos ordenados. En 2 3 4 6 7 8 9 el medio es 6; como 2 < 6, se descarta la derecha. Ahora es seguro: todo lo de la derecha es mayor o igual a 6, así que el 2 no puede estar ahí.

Conclusión. Si los datos no están ordenados hay que ordenarlos antes, o usar búsqueda secuencial. La binaria sobre datos desordenados no avisa del error: devuelve un resultado equivocado.



Binaria: dos ejemplos resueltos

En cada vuelta se calcula el medio, se compara datos[medio] con el valor buscado y se mueve inicio o fin.

Buscar el 7 en 2 3 4 6 7 8 9: lo encuentra en 3 vueltas. La búsqueda secuencial habría necesitado 5 comparaciones.

Buscar el 5 (no está): el tramo se va achicando hasta que inicio supera a fin. Ese cruce de índices significa que no queda ningún candidato, y se devuelve -1.

Dónde iría el 5: entre el 4 y el 6, en la posición 3. Es la información que usa binarySearch de Java (más adelante).

Buscar el 7 en 2 3 4 6 7 8 9

Vuelta	inicio	fin	medio	datos[medio]	Decisión
1	0	6	3	6	7 > 6 → inicio = 4
2	4	6	5	8	7 < 8 → fin = 4
3	4	4	4	7	**¡encontrado!**

Buscar el 5 en 2 3 4 6 7 8 9

Vuelta	inicio	fin	medio	datos[medio]	Decisión
1	0	6	3	6	5 < 6 → fin = 2
2	0	2	1	3	5 > 3 → inicio = 2
3	2	2	2	4	5 > 4 → inicio = 3
4	3	2	—	—	**inicio > fin → -1**

Búsqueda binaria: el código

inicio y fin delimitan el tramo candidato; al principio es todo el arreglo.

while (inicio <= fin) sigue mientras quede al menos un candidato. Cuando inicio supera a fin, el tramo quedó vacío y se devuelve -1.

medio se calcula como inicio + (fin − inicio) / 2 y no como (inicio + fin) / 2, porque esa suma puede pasarse del máximo de un int y dar un número negativo.

medio + 1 / medio − 1 el medio ya se comparó, por eso se lo salta. Error típico: escribir inicio = medio, que puede producir un bucle infinito.

Cuándo termina. O lo encuentra (return medio) o el tramo se agota (return -1). Cada vuelta descarta la mitad de los candidatos.

Busqueda.java

```
1  int busquedaBinaria(int[] datos,
2  int buscado) {
3    int inicio = 0;
4    int fin = datos.length - 1;
5    while (inicio <= fin) {
6      int medio = inicio + (fin - inicio) /
7  2;
8      if (datos[medio] == buscado)
9  return medio;
10     if (datos[medio] < buscado) inicio
11 = medio + 1;
12     else fin = medio - 1;
13   }
14   return -1;
15 }
16
17
18
```

Búsqueda binaria recursiva

Misma idea, otra forma. En lugar de un bucle, la función se llama a sí misma con el tramo ya reducido.

Caso base. Si ini > fin el tramo está vacío: el valor no está y se devuelve -1.

Caso recursivo. Se calcula el medio. Si es el buscado, se devuelve. Si no, se busca solo en la mitad que corresponde, con una nueva llamada.

Llamadas al buscar el 7:

buscar(a, 7, 0, 6) medio = 3 → 7 > 6

buscar(a, 7, 4, 6) medio = 5 → 7 < 8

buscar(a, 7, 4, 4) medio = 4 → igual → 4

Relación con árboles. Es lo mismo que buscar en un árbol binario de búsqueda: en cada paso queda una sola rama por explorar.

Busqueda.java

```
1  int buscar(int[] a, int x, int ini, int
2  fin) {
3    if (ini > fin) return -1;
4    int medio = ini + (fin - ini) / 2;
5    if (a[medio] == x) return medio;
6    if (a[medio] < x) return buscar(a,
7  x, medio + 1, fin);
8    return buscar(a, x, ini, medio -
9  1);
10 }
11
12
```

https://www.geeksforgeeks.org/java/recursion-in-java/

Búsqueda binaria: ¿cuánto cuesta?

La pregunta. ¿Cuántas veces se puede partir n por la mitad hasta llegar a 1? Con 1.000 datos: 1000 → 500 → 250 → 125 → 62 → 31 → 15 → 7 → 3 → 1. Son unas 10 comparaciones en el peor caso.

Logaritmo. Esa cuenta es el logaritmo en base 2 (log2 n). El peor caso de la búsqueda binaria es la parte entera de log2 n, más 1 comparación.

Complejidad. O(log n): si los datos se duplican, hace una sola comparación más.

Comparación. Con un millón de datos, la secuencial hace hasta 1.000.000 de comparaciones y la binaria, 20. La diferencia crece cuantos más datos hay.

Comparaciones en el peor caso

Datos (n)	Secuencial	Binaria
10	10	4
1.000	1.000	10
100.000	100.000	17
1.000.000	1.000.000	20

Secuencial vs. binaria: cuál usar

Secuencial: no exige orden, es simple y siempre correcta; O(n).

Binaria: exige datos ordenados; O(log n).

¿Conviene ordenar para buscar?

Una sola búsqueda: no. Ordenar cuesta más que recorrer una vez; conviene la secuencial.

Muchas búsquedas: sí. Se ordena una sola vez y cada búsqueda cuesta O(log n).

Criterio de elección. Depende de cuántas veces se va a buscar y de si los datos ya vienen ordenados (o se mantienen ordenados).

	Secuencial	Binaria
¿Datos ordenados?	No hace falta	Sí, obligatorio
Estrategia	Uno por uno	Descarta mitades
Mejor caso	O(1)	O(1)
Peor caso	O(n)	O(log n)
Preparación previa	Ninguna	Ordenar: O(n log n)

¿Qué significa ordenar?

Definición. Ordenar es reacomodar los elementos según un criterio de comparación: nota, nombre, fecha, precio… El orden puede ser ascendente (de menor a mayor) o descendente (de mayor a menor).

Ejemplo. 8 3 7 4 9 2 6 queda 2 3 4 6 7 8 9 en forma ascendente, y 9 8 7 6 4 3 2 en forma descendente.

Estabilidad. Un algoritmo es estable si los elementos que empatan en el criterio conservan el orden relativo que tenían antes de ordenar.

Ejemplo de estabilidad. Una lista de alumnos está ordenada por nombre: Ana (8), Luis (8), Marta (6). Si la ordenamos por nota con un algoritmo estable, Ana sigue antes que Luis: Marta 6, Ana 8, Luis 8. Con uno no estable podría quedar Marta 6, Luis 8, Ana 8.

Para qué sirve. Permite ordenar primero por un criterio y después por otro (por nombre y luego por nota) sin perder el primer orden entre los empatados.



Cinco estrategias para el mismo problema

Simples (O(n²)): comparan y mueven de a poco. Son fáciles de entender, pero lentos con muchos datos.

Burbuja: compara vecinos y los intercambia si están al revés; el mayor «flota» hacia el final.

Selección: busca el menor de lo que falta y lo coloca en su posición definitiva.

Inserción: toma cada elemento y lo inserta en su lugar dentro de la parte que ya está ordenada.

Que dividen el problema (O(n log n)): parten el arreglo en partes más chicas, las resuelven y combinan.

Merge Sort: divide por la mitad, ordena cada parte y las intercala.

Quick Sort: parte el arreglo alrededor de un pivote y repite con cada lado.

Costo en el peor caso

Burbuja	**O(n²)**
Selección	**O(n²)**
Inserción	**O(n²)**
Merge Sort	**O(n log n)**
Quick Sort	**O(n²) ***

** Quick Sort es O(n log n) en promedio; su peor caso, O(n²), es raro.*

Burbuja: la idea y una pasada

Idea. Se recorre el arreglo comparando cada elemento con su vecino de la derecha; si están al revés, se intercambian. Así, en cada pasada, el mayor va «flotando» hasta el final, como una burbuja.

Una pasada completa sobre 8 3 7 4 9 2 6:

8 y 3 → 8 > 3, intercambio → 3 8 7 4 9 2 6

8 y 7 → 8 > 7, intercambio → 3 7 8 4 9 2 6

8 y 4 → 8 > 4, intercambio → 3 7 4 8 9 2 6

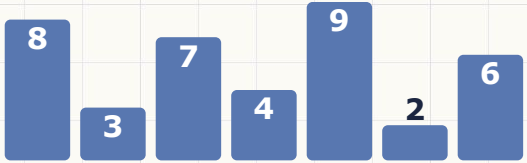
8 y 9 → 8 < 9, no se toca → 3 7 4 8 9 2 6

9 y 2 → 9 > 2, intercambio → 3 7 4 8 2 9 6

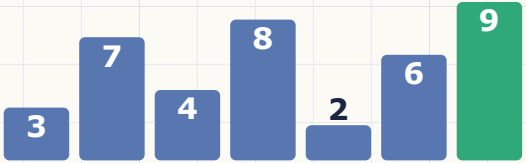
9 y 6 → 9 > 6, intercambio → 3 7 4 8 2 6 9

Resultado. Después de una pasada, el 9 está en su posición definitiva: las pasadas siguientes pueden ignorarlo.

Antes



Después de una pasada



Burbuja: todas las pasadas y su costo

Estado del arreglo al terminar cada pasada:

Inicio: 8 3 7 4 9 2 6

Pasada 1: 3 7 4 8 2 6 9 (5 intercambios)

Pasada 2: 3 4 7 2 6 8 9 (3)

Pasada 3: 3 4 2 6 7 8 9 (2)

Pasada 4: 3 2 4 6 7 8 9 (1)

Pasada 5: 2 3 4 6 7 8 9 (1)

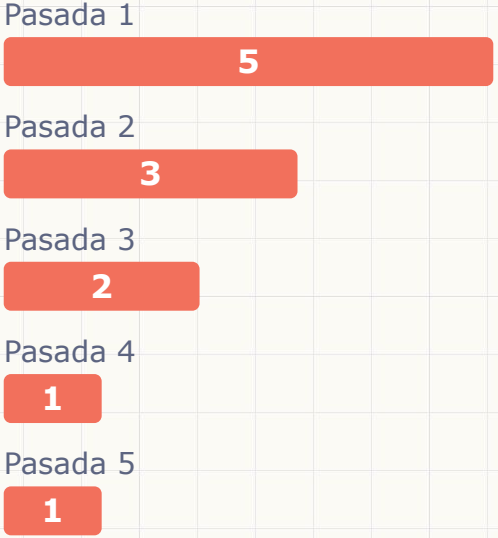
Pasada 6: sin cambios ⇒ termina

Cantidad de pasadas. Con n elementos hacen falta como máximo n − 1 pasadas (acá, 6); la última no intercambia nada.

Costo. En la pasada k se comparan n − k pares: (n − 1) + (n − 2) + … + 1 = n(n − 1)/2 comparaciones. Son 21 para n = 7 y casi 500.000 para n = 1.000. Por eso es O(n²).

Dato curioso. Se hicieron 12 intercambios, justo la cantidad de pares «al revés» del arreglo original: cada intercambio de vecinos arregla exactamente un par.

Intercambios por pasada



Total: 12 intercambios y 21 comparaciones.

Burbuja: el código y la bandera

for externo cuenta las pasadas; cada una deja el mayor del tramo en su lugar.

for interno compara vecinos. El − i evita revisar lo que ya quedó ordenado al final.

Intercambio. Se usa una variable auxiliar (aux) para no perder un valor.

Bandera huboCambio. Si una pasada no intercambió nada, el arreglo ya está ordenado y se corta con break. Con 1 2 3 5 4 termina en 2 pasadas en vez de 4.

Costo. Peor caso O(n²). Mejor caso (ya ordenado): O(n), gracias a la bandera, con una sola pasada de comprobación. Es estable.

Ordenamiento.java

```java
void burbuja(int[] datos) {
  for (int i = 0; i < datos.length - 1; i++) {
    boolean huboCambio = false;
    for (int j = 0; j < datos.length - 1 - i; j++) {
      if (datos[j] > datos[j + 1]) {
        int aux = datos[j];
        datos[j] = datos[j + 1];
        datos[j + 1] = aux;
        huboCambio = true;
      }
    }
    if (!huboCambio) break;
  }
}
```

Selección: la idea y un ejemplo

Idea. En cada pasada se busca el menor elemento de la parte que todavía no está ordenada y se lo intercambia con el que está en la primera posición de esa parte. Es como armar una fila por altura: se elige al más bajo, se lo manda primero y se repite con los que quedan.

Propiedad clave. Después de cada pasada, un elemento queda en su posición definitiva y ya no se vuelve a tocar.

Ejemplo sobre 8 3 7 4 9 2 6 (a la derecha, el arreglo al terminar la pasada):

Pasada 1: menor 2 (pos. 5) ⇔ pos. 0 ⇒ 2 3 7 4 9 8 6

Pasada 2: menor 3, ya en su lugar ⇒ 2 3 7 4 9 8 6

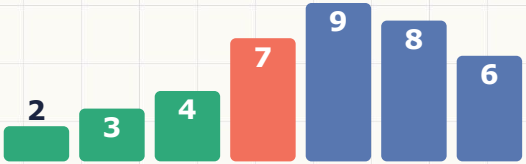
Pasada 3: menor 4 (pos. 3) ⇔ pos. 2 ⇒ 2 3 4 7 9 8 6

Pasada 4: menor 6 (pos. 6) ⇔ pos. 3 ⇒ 2 3 4 6 9 8 7

Pasada 5: menor 7 (pos. 6) ⇔ pos. 4 ⇒ 2 3 4 6 7 8 9

Pasada 6: menor 8, ya en su lugar ⇒ 2 3 4 6 7 8 9

Al terminar la pasada 3



Al terminar la pasada 5



Verde: parte ya definitiva. Coral: elemento con el que se intercambió.

Selección: el código

for externo i es la posición donde debe ir el menor de lo que queda.

posMin guarda la posición (no el valor) del menor encontrado hasta ahora; arranca en i.

for interno recorre el resto y actualiza posMin cada vez que encuentra un valor menor.

Intercambio. Al terminar el for interno se intercambia datos[i] con datos[posMin].

Costo. Siempre n(n − 1)/2 comparaciones (21 para n = 7), aunque el arreglo ya esté ordenado, porque debe buscar el menor igual. Pero hace pocos intercambios: como máximo n − 1 (acá, 4). Es útil cuando mover datos es costoso.

No es estable.

ordenamiento.java

```
1  void seleccion(int[] datos) {
2    for (int i = 0; i < datos.length - 1; i++) {
3      int posMin = i;
4      for (int j = i + 1; j < datos.length; j++) {
5        if (datos[j] < datos[posMin]) {
6          posMin = j;
7        }
8      }
9      int aux = datos[i];
10     datos[i] = datos[posMin];
11     datos[posMin] = aux;
12   }
13 }
```

Inserción: la idea y un ejemplo

Idea. Es como ordenar las cartas que se tienen en la mano: se toma una carta nueva y se la inserta en su lugar dentro de las que ya están ordenadas, corriendo las mayores un lugar hacia la derecha.

Propiedad clave. La parte izquierda siempre está ordenada, pero no es definitiva: un elemento nuevo puede obligar a correr varios.

Ejemplo sobre 8 3 7 4 9 2 6 (entre corchetes, la parte ordenada):

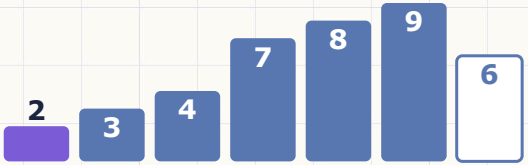
[8] 3 7 4 9 2 6	inicio
[3 8] 7 4 9 2 6	inserto 3: corro el 8 (1)
[3 7 8] 4 9 2 6	inserto 7: corro el 8 (1)
[3 4 7 8] 9 2 6	inserto 4: corro 8 y 7 (2)
[3 4 7 8 9] 2 6	inserto 9: ya está en su lugar (0)
[2 3 4 7 8 9] 6	inserto 2: corro 5 elementos (5)
[2 3 4 6 7 8 9]	inserto 6: corro 9, 8 y 7 (3)

Total. 12 desplazamientos y 16 comparaciones.

Al inicio



Al insertar el 2



Azul: parte ordenada. Violeta: el elemento que se inserta. Borde: sin procesar.

Inserción: el código

for arranca en la posición 1, porque el primer elemento solo ya es una parte ordenada.

clave es el elemento a insertar. Se guarda aparte porque su posición se va a pisar al correr los demás.

j recorre la parte ordenada de derecha a izquierda.

while mientras el elemento de la izquierda sea mayor que la clave, se lo corre una posición a la derecha.

datos[j + 1] = clave al terminar el while queda un hueco justo donde corresponde, y ahí va la clave.

Costo. Peor caso O(n²). Pero si el arreglo ya está ordenado (o casi), el while casi no se ejecuta: n − 1 comparaciones, O(n). Es la mejor de las tres con datos casi ordenados, y es estable.

ordenamiento.java

```
1  void insercion(int[] datos) {
2    for (int i = 1; i < datos.length; i++) {
3      int clave = datos[i];
4      int j = i - 1;
5      while (j >= 0 && datos[j] > clave)
6  {
7        datos[j + 1] = datos[j];
8        j--;
9      }
10     datos[j + 1] = clave;
11   }
   }
```

Los tres algoritmos simples, comparados

Los tres son O(n²) en el peor caso: sirven para entender cómo se ordena, no para millones de datos.

Burbuja: muy didáctica; con la bandera detecta rápido un arreglo ya ordenado.

Selección: pocas escrituras (a lo sumo n − 1 intercambios); útil si mover datos es caro.

Inserción: la mejor de las tres con datos casi ordenados o arreglos chicos.

Con nuestros datos (8 3 7 4 9 2 6) las comparaciones y los movimientos fueron los de la tabla.

	Burbuja	Selección	Inserción
Comparaciones (n = 7)	21	21	16
Movimientos	12 intercambios	4 intercambios	12 desplazamientos
Mejor caso	O(n) con bandera	O(n²)	O(n)
Peor caso	O(n²)	O(n²)	O(n²)
¿Estable?	Sí	No	Sí

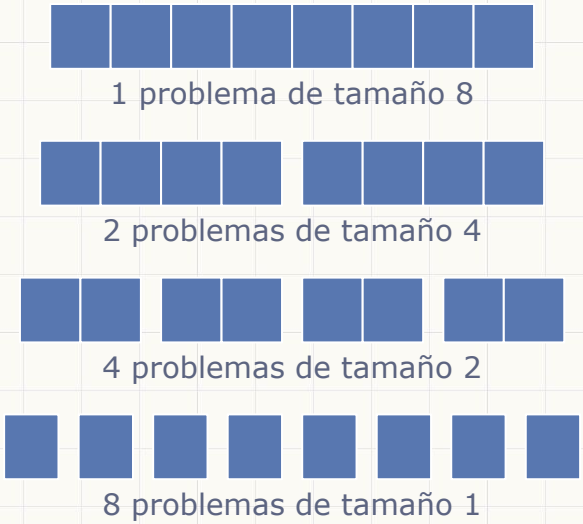
Dividir para vencer

Idea. Un problema grande se parte en problemas más chicos del mismo tipo, se resuelven por separado y se combinan los resultados. Es como repartir 1.000 exámenes entre varios docentes: cada uno ordena su pila y después se juntan intercalando.

Por qué es más rápido. Si se divide a la mitad cada vez, hay alrededor de log2 n niveles. En cada nivel se trabaja, en total, alrededor de n. El costo total es n × log2 n.

Comparación. Con n = 1.000, n² son un millón de pasos y n × log2 n son unos 10.000.

Dos algoritmos que lo usan: Merge Sort (divide fácil y trabaja al combinar) y Quick Sort (trabaja al dividir y no necesita combinar).



Intercalación: unir dos secuencias ordenadas

Problema. Dadas dos secuencias ya ordenadas, obtener una sola, también ordenada, con todos sus elementos.

Idea. Como dos filas que se unen en una (cierre de cremallera), o dos mazos ordenados: se mira el primero de cada uno y se pasa el menor al resultado. Se repite hasta que uno se agota; después se copia lo que queda del otro.

Ejemplo con A = [2, 5, 8, 11] y B = [3, 4, 9]:

2 vs 3 ⇒ paso el 2 (de A) R = 2
5 vs 3 ⇒ paso el 3 (de B) R = 2 3
5 vs 4 ⇒ paso el 4 (de B) R = 2 3 4
5 vs 9 ⇒ paso el 5 (de A) R = 2 3 4 5
8 vs 9 ⇒ paso el 8 (de A) R = 2 3 4 5 8
11 vs 9 ⇒ paso el 9 (de B) R = 2 3 4 5 8 9
B se agotó ⇒ copio el resto de A (11) R = 2 3 4 5 8 9 11

Costo. Cada elemento se copia una sola vez: O(n + m).



Azul: viene de A. Turquesa: viene de B.

Intercalación: el código

Tres índices que solo avanzan: i recorre A, j recorre B y k va llenando el resultado r.

Primer while mientras ambas listas tengan elementos, copia el menor. El <= hace que, ante un empate, salga primero el de A, y así los iguales conservan su orden (estabilidad).

Los otros dos while copian lo que sobra de la lista que no se agotó; solo uno de los dos hace algo.

r[k++] = a[i++] significa «copiá a[i] en r[k] y avanzá los dos índices».

Uso. Es la pieza central de Merge Sort, pero sirve por sí sola para unir dos secuencias ordenadas (por ejemplo, dos listas de alumnos por nota).

Costo. O(n + m): cada elemento se copia una sola vez.

intercalacion.java

```
int[] intercalar(int[] a, int[] b) {
  int[] r = new int[a.length + b.length];
  int i = 0, j = 0, k = 0;
  while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) r[k++] = a[i++];
    else r[k++] = b[j++];
  }
  while (i < a.length) r[k++] = a[i++];
  while (j < b.length) r[k++] = b[j++];
  return r;
}
```

Merge Sort: la idea y un ejemplo

Idea. Se divide el arreglo por la mitad una y otra vez hasta que cada parte tiene un solo elemento (que ya está ordenado), y después se intercalan las partes de a pares, subiendo nivel por nivel.

Dividir:

[8 3 7 4 9 2 6]

[8 3 7 4] [9 2 6]

[8 3] [7 4] [9 2] [6]

[8] [3] [7] [4] [9] [2] [6]

Combinar (intercalando):

[3 8] [4 7] [2 9] [6]

[3 4 7 8] [2 6 9]

[2 3 4 6 7 8 9]

Niveles. Con n = 7 hay 3 niveles de división y 3 de mezcla. En cada nivel se trabaja alrededor de n.



Arriba: dividir. Abajo: combinar (verde = parte ya ordenada).

Merge Sort: el código

Caso base. Si inicio ≥ fin, el tramo tiene 0 o 1 elemento y ya está ordenado.

Recursión. Se calcula el medio, se ordena la mitad izquierda y se ordena la mitad derecha.

mezclar. Con las dos mitades ya ordenadas, las combina en un único tramo ordenado (es la intercalación).

Orden de ejecución. mergeSort baja hasta los tramos de un elemento y recién ahí empiezan las mezclas, de abajo hacia arriba.

Costo. Siempre O(n log n): hay log n niveles y en cada uno se mezclan n elementos. Es estable.

Memoria. Necesita un arreglo auxiliar de tamaño n para mezclar: O(n) de memoria extra. Es el precio de la garantía de tiempo.

mergeSort.java

```
1 void mergeSort(int[] datos, int
inicio, int fin) {
2   if (inicio < fin) {
3     int medio = inicio + (fin - inicio) /
4 2;
5     mergeSort(datos, inicio,
medio);
6     mergeSort(datos, medio + 1, fin);
7     mezclar(datos, inicio, medio,
8 fin);
    }
}
```

Merge Sort: el código de mezclar

aux arreglo auxiliar donde se arma el tramo mezclado.

Primer while igual que en la intercalación: copia el menor de los dos tramos, d[ini..medio] y d[medio+1..fin].

Siguientes while copian lo que queda de uno de los tramos.

for final copia el resultado desde aux de vuelta al arreglo original, en las posiciones ini..fin.

Relación. mezclar es el mismo intercalar de antes, pero trabajando sobre dos tramos contiguos del mismo arreglo en lugar de dos arreglos separados.

Costo. Mezclar un tramo de tamaño t cuesta O(t). Sumando todos los tramos de un nivel, cada nivel cuesta O(n).

mergesort.java

```
1  void mezclar(int[] d, int ini, int medio,
2  int fin) {
3    int[] aux = new int[fin - ini + 1];
4    int i = ini, j = medio + 1, k = 0;
5    while (i <= medio && j <= fin) {
6      if (d[i] <= d[j]) aux[k++] = d[i++];
7      else aux[k++] = d[j++];
8    }
9    while (i <= medio) aux[k++] = d[i++];
10   while (j <= fin) aux[k++] = d[j++];
11   for (k = 0; k < aux.length; k++) {
12     d[ini + k] = aux[k];
13   }
}
```

Quick Sort: la idea y el pivote

Idea. Se elige un elemento como pivote. Se reacomoda el arreglo para que todos los menores queden a su izquierda y todos los mayores a su derecha. Con eso, el pivote queda en su posición definitiva. Después se repite el mismo proceso con cada lado.

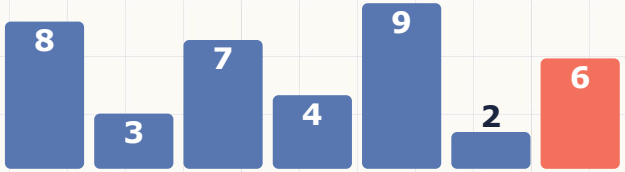
Partición. Es el paso clave: reorganizar el arreglo alrededor del pivote. Cuesta una sola pasada lineal, O(n).

Ejemplo. En 8 3 7 4 9 2 6 con pivote 6, la partición deja 3 4 2 | 6 | 9 7 8. El 6 ya está en el índice 3; falta ordenar 3 4 2 y 9 7 8.

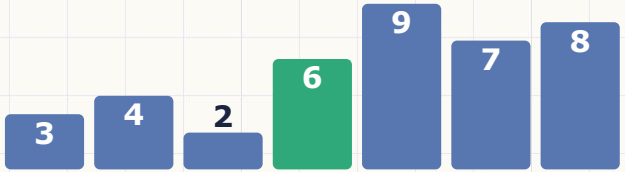
Elección del pivote. Puede ser el primero, el último, uno al azar o la mediana de tres. Acá usamos el último elemento.

Diferencia con Merge Sort. Quick Sort hace el trabajo al dividir (la partición) y no necesita combinar; Merge Sort divide fácil y trabaja al combinar.

Antes (pivote = 6, el último)



Después de particionar



menores | pivote | mayores

Quick Sort: la partición paso a paso

Dos índices. j recorre el arreglo. i marca la «frontera» de la zona de menores: todo lo que está hasta i es menor que el pivote. Arranca en −1 (zona vacía).

Regla. Si d[j] es menor que el pivote, la frontera avanza (i++) y se intercambia d[i] con d[j]. Si no, no se hace nada.

Ejemplo con pivote 6 sobre 8 3 7 4 9 2 6:

j=0: 8 no es menor que 6 ⇒ nada

j=1: 3 es menor ⇒ i=0, d[0]⇔d[1] ⇒ 3 8 7 4 9 2 6

j=2: 7 no es menor ⇒ nada

j=3: 4 es menor ⇒ i=1, d[1]⇔d[3] ⇒ 3 4 7 8 9 2 6

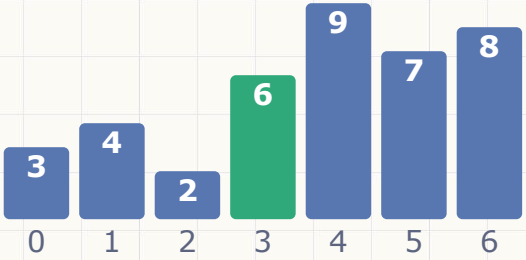
j=4: 9 no es menor ⇒ nada

j=5: 2 es menor ⇒ i=2, d[2]⇔d[5] ⇒ 3 4 2 8 9 7 6

Final: pivote a la posición i+1 ⇒ 3 4 2 6 9 7 8

Resultado. El 6 quedó en el índice 3: los menores (3 4 2) a la izquierda y los mayores (9 7 8) a la derecha.

Resultado de la partición



El pivote (verde) ya no se mueve más.

Quick Sort: el código de particionar

pivote se toma el último elemento del tramo.

i empieza en ini − 1: la zona de menores está vacía.

for / if cada d[j] menor que el pivote hace crecer la zona (i++) y se intercambia con d[i].

Después del for el pivote (que está en d[fin]) se intercambia con d[i + 1]: queda justo entre los menores y los mayores.

return i + 1 devuelve la posición definitiva del pivote; quickSort la usa para dividir el trabajo.

Costo. Una sola pasada: O(n) comparaciones.

QuickSort.java

```
1  int particionar(int[] d, int ini, int fin) {
2    int pivote = d[fin];
3    int i = ini - 1;
4    for (int j = ini; j < fin; j++) {
5      if (d[j] < pivote) {
6        i++;
7        int aux = d[i]; d[i] = d[j]; d[j] = aux;
8      }
9    }
10   int aux = d[i + 1]; d[i + 1] = d[fin]; d[fin] = aux;
11   return i + 1;
12 }
13
14
15
```

Quick Sort: recursión, costo y peor caso

Caso base. Con 0 o 1 elemento (ini ≥ fin) el tramo ya está ordenado.

Recursión. Se particiona; el pivote queda fijo; se ordenan por separado el tramo de la izquierda y el de la derecha.

Costo normal. Si el pivote parte el arreglo más o menos por la mitad, hay unos log n niveles de llamadas y el costo es O(n log n).

Peor caso. Si el arreglo ya está ordenado y el pivote es siempre el último, el pivote es el mayor: todo queda a la izquierda y nada a la derecha. La partición no divide: hay n niveles y el costo pasa a O(n²). Es el mismo problema de un árbol de búsqueda que degenera en una lista.

Cómo evitarlo. Elegir el pivote al azar o la mediana de tres (primero, medio y último).

Memoria. No necesita arreglo auxiliar, solo la pila de llamadas (O(log n) en promedio). No es estable.

quicksort.java

```
void quicksort(int[] a, int ini, int
fin) {
  if (ini < fin) {
    int p = particionar(a, ini, fin);
    quicksort(a, ini, p - 1);
    quicksort(a, p + 1, fin);
  }
}
```

Caso	Qué hace el pivote	Costo
Normal	Parte el arreglo más o menos por la mitad	O(n log n)
Peor	Deja todo de un lado (arreglo ordenado, pivote = último)	**O(n²)**

Merge Sort vs. Quick Sort

Merge Sort garantiza O(n log n) y es estable, pero usa memoria extra (un arreglo auxiliar).

Quick Sort casi no usa memoria extra y suele ser más rápido en la práctica, pero tiene un peor caso $O(n^2)$ y no es estable.

¿Cuál elegir? Depende de si importan la estabilidad, la memoria o la garantía en el peor caso. No hay un ganador absoluto.

	Merge Sort	Quick Sort
Estrategia	Divide en mitades y combina intercalando	Parte alrededor de un pivote; no combina
Mejor / promedio	O(n log n)	O(n log n)
Peor caso	O(n log n), siempre	**$O(n^2)$ con mal pivote**
Memoria extra	O(n): arreglo auxiliar	O(log n): pila de recursión
¿Estable?	Sí	No
Cuándo conviene	Garantía de tiempo y estabilidad	Arreglos grandes en memoria

Complejidad de los cinco algoritmos

Cómo leer la tabla. Mejor, promedio y peor son los tres escenarios de costo. Memoria extra es el espacio adicional que necesita el algoritmo. Estable indica si respeta el orden de los empatados.

Verde marca crecimiento moderado; rosa, crecimiento muy rápido con n.

Para tener en cuenta: la memoria extra y la estabilidad también cuentan al elegir un algoritmo, no solo el tiempo.

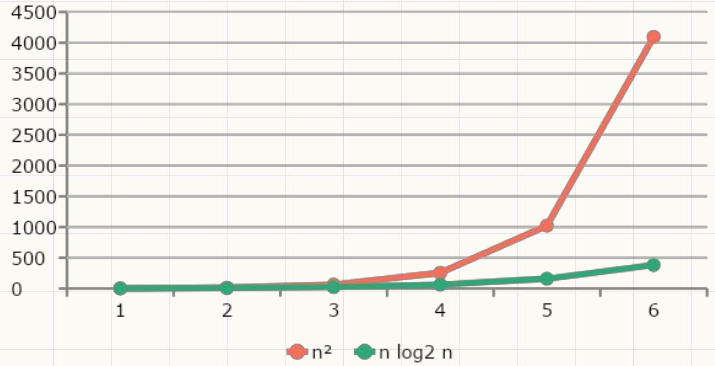
Algoritmo	Mejor	Promedio	Peor	Memoria	Estable
Burbuja (con bandera)	O(n)	O(n²)	O(n²)	O(1)	Sí
Selección	O(n²)	O(n²)	O(n²)	O(1)	No
Inserción	O(n)	O(n²)	O(n²)	O(1)	Sí
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Sí
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No

Por qué importa la complejidad

No es «un poco más rápido». Con un millón de datos, un algoritmo O(n²) hace del orden de un billón de pasos; uno O(n log n), unos 20 millones.

En tiempo (ilustrativo). Aunque cada paso tardara una milmillonésima de segundo, un billón de pasos serían unos 17 minutos; 20 millones, apenas 0,02 segundos.

Conclusión. La elección del algoritmo es lo que separa un programa usable de uno que no termina. Para pocos datos da lo mismo; para muchos, define todo.



n	n²	n log2 n
1.000	1.000.000	≈ 10.000
100.000	10.000.000.000	≈ 1,7 millones
1.000.000	1 billón	≈ 20 millones

Buscar objetos: == contra equals

Datos reales. En un programa se buscan y ordenan objetos (alumnos, productos, clientes), no solo números. Usamos la clase Alumno de la derecha.

== contra equals. == pregunta «¿es el mismo objeto?» (la misma referencia). equals pregunta «¿tienen el mismo contenido?».

```
String a = new String("Ana");
String b = new String("Ana");
a == b      // false: dos objetos distintos
a.equals(b) // true: mismo contenido
```

Búsqueda de un objeto. Es la misma búsqueda secuencial de antes; solo cambia qué se compara:

```
if (lista.get(i).getNombre().equals(nombre)) return i;
```

Sobrescribir equals. Para que equals compare un Alumno completo, la clase debe sobrescribirlo e indicar qué significa «mismo alumno» (por ejemplo, el mismo DNI). Por defecto, equals se comporta igual que ==.

Alumno.java

```
1  class Alumno {
2    private String nombre;
3    private int nota;
4
5    public Alumno(String nombre, int
6  nota) {
7      this.nombre = nombre;
8      this.nota = nota;
9    }
10   public String getNombre() {
11 return nombre; }
12   public int getNota() { return nota;
13 }
   }
```

Ordenar objetos: Comparable

El problema. Con números, Java sabe qué es «menor». Con dos Alumno no puede adivinarlo: ¿se ordenan por nota, por nombre, por edad? Si hacemos Collections.sort(lista) sin darle una regla, ni siquiera compila.

Comparable. Es una interfaz: una promesa. Al implementar Comparable<**Alumno**>, la clase se compromete a saber compararse con otro Alumno mediante compareTo.

compareTo. this es el **alumno** que llama y **otro** es con quien se compara. Devuelve un número negativo si this va antes, cero si son iguales y positivo si this va después. Ej.: Ana (8) contra Luis (6) da positivo: Ana va después.

Integer.compare se usa en lugar de restar las notas, porque la resta puede fallar con números muy grandes.

Cómo lo usa sort. Es como una persona que sabe ordenar pero no sabe qué es «menor» para un Alumno: cada vez que necesita saber cuál de dos va antes, le pregunta al objeto con compareTo. Por polimorfismo, el mismo sort sirve para String, Integer o Alumno.

Limitación. Una clase tiene un solo compareTo: un solo orden natural.

Alumno.java

```
class Alumno implements
Comparable<Alumno> {
  private String nombre;
  private int nota;
  // constructor y getters...

  @Override
  public int compareTo(Alumno
otro) {
    return
Integer.compare(this.nota,
otro.nota);
Collections.sort(lista);  // ordena por nota
Arrays.sort(alumnos);    // arreglo de
Alumno
```

Otros criterios: Comparator

Para qué. Para ordenar por otro criterio sin modificar la clase, o para tener varios criterios. Se crea aparte un objeto que sabe comparar dos alumnos.

Comparable vs. Comparator. compareTo está dentro de la clase y dice «yo me comparo con otro» (un solo orden natural). compare está afuera y dice «comparen a estos dos» (tantos criterios como se necesiten: PorNombre, PorNota, PorEdad…).

Uso. lista.sort(new PorNombre()); El algoritmo no cambia: solo cambia el criterio de comparación que se le entrega.

Resultado con Ana 8, Luis 6, Marta 9 y Juan 6:

Por nota: Luis 6, Juan 6, Ana 8, Marta 9

Por nombre: Ana 8, Juan 6, Luis 6, Marta 9

Estabilidad. Luis sigue antes que Juan (misma nota): se conserva el orden de los iguales.

Búsqueda binaria con objetos. Funciona igual, pero la lista debe estar ordenada con el mismo criterio con el que se busca.

PorNombre.java

```
class PorNombre implements
Comparator<Alumno> {
  @Override
  public int compare(Alumno a,
Alumno b) {
    return
a.getNombre().compareTo(b.getNo
mbre());
  }
}

lista.sort(new PorNombre());
// Java moderno permite escribirlo
más corto con lambdas
```

Lo que ya trae Java

En la práctica no se programan estos algoritmos a mano: Java ya los trae.

Arrays.sort(int[]). Ordena arreglos de primitivos; usa una variante de Quick Sort (Dual-Pivot).

Collections.sort(lista) y lista.sort(c). Ordenan objetos con Comparable o Comparator; usan TimSort, una variante de Merge Sort que es estable.

binarySearch. Arrays.binarySearch y Collections.binarySearch hacen la búsqueda binaria y exigen datos ordenados. Devuelven el índice, o −(punto de inserción) − 1 si el valor no está.

Idea final. Usar una función ya hecha no reemplaza entender la estrategia, su costo y sus condiciones (por ejemplo, que los datos estén ordenados).

Main.java

```
1 int[] datos = {2, 3, 4, 6, 7, 8, 9};
2 Arrays.binarySearch(datos, 7);  // 4
3 Arrays.binarySearch(datos, 5);  //
4 -4
  // el 5 iría en la posición 3: -(3) - 1 = -4
```

Herramienta	Por dentro
Arrays.sort(int[])	Dual-Pivot Quick Sort
Collections.sort / lista.sort	TimSort (variante de Merge Sort, estable)
binarySearch	Búsqueda binaria

¿Qué estrategia elegir?

La pregunta no es «¿cuál sé implementar?», sino «¿qué datos tengo y qué necesito hacer con ellos?»

Qué mirar para decidir:

- cuántos datos hay;
- si ya están ordenados o se mantienen ordenados;
- cuántas veces se va a buscar;
- si importa la estabilidad;
- cuánta memoria extra se puede usar.

En un programa real se usa la biblioteca de Java; entender los algoritmos sirve para elegir bien y para saber qué condiciones exigen.

Situación	Conviene
Pocos datos, o una sola búsqueda	Búsqueda secuencial
Muchas búsquedas sobre datos estables	Ordenar una vez + binaria
Datos casi ordenados o arreglos chicos	Inserción
Garantía de tiempo y estabilidad	Merge Sort
Arreglo grande, la estabilidad no importa	Quick Sort
Un programa real	Arrays.sort / Collections.sort

Propuesta para Práctica y entregable

Práctica 1. En grupos, elegir un algoritmo e implementarlo. Explicar la estrategia y mostrar un ejemplo.

Entregable 2. Modificar el entregable anterior para administrar almacenamiento con estructuras dinámicas y aplicar los algoritmos solicitados.

En la explicación hay que responder cuatro preguntas:

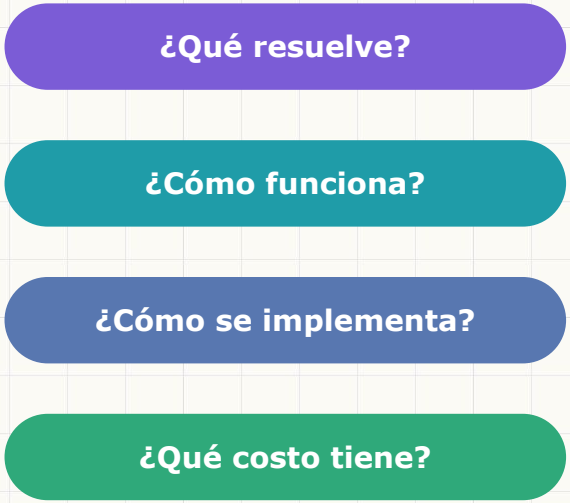
- **¿Qué problema resuelve?**

¿Cómo funciona? Con un ejemplo resuelto paso a paso.

¿Cómo se implementa? El código y qué hace cada parte.

¿Qué costo tiene? Mejor y peor caso, en comparaciones.

Recomendación. No alcanza con mostrar que el código funciona: hay que poder explicar por qué funciona y cuánto cuesta.



No hay una única estrategia

Para buscar

- Secuencial: recorrer; sirve siempre, O(n).
- Binaria: descartar mitades; exige datos ordenados, O(log n).
- Con muchas búsquedas, ordenar primero puede ser la mejor inversión.

Para ordenar

- Burbuja, selección e inserción: simples, $O(n^2)$.
- Merge Sort y Quick Sort: dividen el problema, O(n log n).
- En Java: Arrays.sort y Collections.sort; con objetos, Comparable o Comparator.

